

DEVELOPER CERTIFICATION STUDY GUIDE

Anthropic Claude Developer Certification Prep

Master the Claude API, SDK patterns, streaming, tool use, prompt caching, batch processing, structured outputs, and production deployment. Includes code examples in Python and TypeScript, architecture diagrams, and 50 developer-focused MCQs with explanations.

Exam: CCDV-F (Developer Foundations) **Sections:** 12 **MCQs:** 50 questions **Source:** Official Anthropic Docs

Updated: September 2026

Developer-Level Guide

This guide is designed for developers building production applications with the Claude API and SDKs. It assumes familiarity with REST APIs, JSON, and at least one programming language (Python or TypeScript examples provided). For foundational concepts (model families, pricing overview, safety principles), see the companion Associate-level study guide.

TABLE OF CONTENTS

01 Developer Ecosystem Overview	02 Messages API Deep Dive
03 Streaming & Real-Time Responses	04 Tool Use & Function Calling
05 Structured Outputs & JSON Mode	06 Prompt Caching

[07 Batch Processing](#)

[09 Rate Limits, Errors & Resilience](#)

[11 Security, Data & Production](#)

[08 Thinking & Reasoning](#)

[10 Testing, Evaluation & QA](#)

[12 Practice Test — 50 Developer MCQs](#)

PART 01

Developer Ecosystem Overview

Three Building Surfaces

Anthropic provides three distinct ways to build with Claude. Choosing the right surface depends on how much control you need versus how much infrastructure you want Anthropic to manage.

Dimension	Claude API + Tool Use	Claude Agent SDK	Managed Agents (CMA)
Control Level	Full — you own the loop	High — SDK provides harness	Managed — Anthropic runs the loop
Hosting	You host everything	You host everything	Anthropic hosts loop + sandbox
Billing	Per-token API usage	Per-token API usage	Per-token + session costs
Tools	You define all tools	Built-in (file, bash, grep) + custom	Sandboxed bash, files, code exec + MCP
State	You manage conversation state	SDK manages context	Persistent sessions with workspaces
Best For	Custom integrations, pipelines	Coding agents on your infra	Long-running async agent tasks
Package	<code>anthropic</code> / <code>@anthropic-ai/sdk</code>	<code>claude-agent-sdk</code>	REST API (beta)

Tool Runner vs Agent SDK

The Tool Runner (`client.beta.messages.tool_runner`) is part of the standard Anthropic SDK and automates the tool-call loop for tools *you* define. The Claude Agent SDK is a separate package that provides the full Claude Code harness with built-in tools. They are different products despite similar names.

Developer Console & SDKs

The **Anthropic Console** (console.anthropic.com) provides API key management, usage monitoring, rate limit dashboards, and a prompt playground (Workbench) for testing API calls interactively.

Official SDKs

Language	Package	Install
Python	<code>anthropic</code>	<code>pip install anthropic</code>
TypeScript / JS	<code>@anthropic-ai/sdk</code>	<code>npm install @anthropic-ai/sdk</code>
Go	<code>anthropic-sdk-go</code>	<code>go get github.com/anthropics/anthropic-sdk-go</code>
Java / Kotlin	<code>com.anthropic:anthropic-java</code>	Maven / Gradle
C#	<code>Anthropic</code>	<code>dotnet add package Anthropic</code>
PHP	<code>anthropic/anthropic-sdk-php</code>	<code>composer require anthropic/anthropic-sdk-php</code>
Ruby	<code>anthropic-sdk-ruby</code>	<code>gem install anthropic-sdk-ruby</code>

All SDKs auto-detect the `ANTHROPIC_API_KEY` environment variable. The CLI tool `ant` supports OAuth login (`ant auth login`), which SDKs detect automatically—no env var needed after login.

API Versioning

- **`anthropic-version` header** — Required on every raw HTTP request. Current stable version defines the response format and available features.
- **Pinned model IDs** — Use exact model strings like `claude-opus-5` , `claude-sonnet-5` . Never append date suffixes.
- **Beta features** — Activated via the `anthropic-beta` header (comma-separated). Example: `anthropic-beta: fast-mode-2026-02-01` . SDKs accept a `betas` array parameter.
- **Models API** — `GET /v1/models` and `GET /v1/models/{id}` for live capability discovery (context window, max output, supported features).

Key Model IDs to Know

- `claude-opus-5` — Primary high-capability model (\$5/\$25 per MTok)
- `claude-sonnet-5` — Balanced speed/quality (\$2/\$10 per MTok)
- `claude-haiku-4-5` — Fastest, lowest cost (\$1/\$5 per MTok, 200K context)
- `claude-fable-5-1` — Most capable, frontier reasoning (\$10/\$50 per MTok)

PART 02

Messages API Deep Dive

Endpoint

Everything goes through `POST /v1/messages` . Tools, structured outputs, thinking, and caching are all features of this single endpoint—not separate APIs.

Full Request Parameters

Parameter	Type	Required	Description
<code>model</code>	string	Yes	Model ID, e.g. <code>"claude-opus-5"</code>
<code>max_tokens</code>	integer	Yes	Maximum output tokens. Up to 128K on current models.
<code>messages</code>	array	Yes	Conversation turns. Alternating <code>user</code> / <code>assistant</code> roles.
<code>system</code>	string array	No	System prompt. Can be a string or array of content blocks.
<code>tools</code>	array	No	Tool definitions (user-defined or server-side).
<code>tool_choice</code>	object	No	Tool selection mode: <code>auto</code> , <code>any</code> , <code>tool</code> , <code>none</code> .
<code>temperature</code>	float	No	Sampling randomness (0.0–1.0). Removed on Opus 4.6+ / Sonnet 5.
<code>top_p</code>	float	No	Nucleus sampling. Removed on newer models.
<code>top_k</code>	integer	No	Top-K sampling. Removed on newer models.
<code>stop_sequences</code>	array	No	Custom stop strings. Generation halts when encountered.
<code>stream</code>	boolean	No	Enable Server-Sent Events streaming.
<code>thinking</code>	object	No	Thinking configuration: <code>adaptive</code> or <code>enabled</code> (legacy).
<code>output_config</code>	object	No	Contains <code>format</code> (structured output) and <code>effort</code> level.
<code>metadata</code>	object	No	Contains <code>user_id</code> for abuse detection and tracking.
<code>cache_control</code>	object	No	Top-level automatic prompt caching.

Multi-Turn Conversation Pattern

```
from anthropic import Anthropic client = Anthropic() conversation = [] def chat(user_message: str) -> str: conversation.append({"role": "user", "content": user_message}) response = client.messages.create( model="claude-opus-5", max_tokens=4096, system="You are a helpful coding assistant.", messages=conversation ) assistant_text = response.content[0].text # Append the full content (not just text) to preserve structure conversation.append({"role": "assistant", "content": response.content}) return assistant_text
```

System Prompts

- Set via the top-level `system` parameter, separate from `messages` .
- System tokens count as **input tokens** and are billed at the input rate.
- Keep stable system prompts first for optimal prompt caching.
- Can be a plain string or an array of content blocks (with `cache_control`).
- Mid-conversation system messages (`"role": "system"` in `messages[]`) are supported on Opus 4.8+, Fable 5+ to inject instructions without invalidating cache.

Content Block Types

Block Type	Direction	Description
<code>text</code>	Both	Plain text content. Most common block type.
<code>image</code>	User → API	Base64-encoded image or URL. Supports PNG, JPEG, GIF, WebP.
<code>document</code>	User → API	PDF or text document. Base64 or Files API reference.
<code>tool_use</code>	API → Client	Claude requests a tool call. Contains <code>id</code> , <code>name</code> , <code>input</code> .
<code>tool_result</code>	Client → API	Result of executing a tool. References <code>tool_use_id</code> .
<code>thinking</code>	API → Client	Reasoning trace. Has <code>thinking</code> text and <code>signature</code> field.

Stop Reasons & Handling

```
response = client.messages.create(model="claude-opus-5", ...) match response.stop_reason: case "end_turn": # Normal completion - read response.content print(response.content[0].text) case "max_tokens": # Output was truncated - consider increasing max_tokens print("Warning: response truncated") case "stop_sequence": # Hit a custom stop sequence print("Stopped at custom delimiter") case "tool_use": # Claude wants to call a tool - enter agentic loop handle_tool_calls(response) case "refusal": # Safety classifier declined (Fable 5.1 / Opus 5+) category = response.stop_details.category print(f"Refused: {category}")
```

Response Usage Object

Every response includes `response.usage` with: `input_tokens` , `output_tokens` , `cache_creation_input_tokens` , and `cache_read_input_tokens` . Use these to track costs and cache performance.

PART 03

Streaming & Real-Time Responses

Server-Sent Events (SSE) Protocol

When `stream: true` is set, the API returns an SSE stream. Each event has an `event:` type line followed by a `data:` JSON payload line. The stream delivers tokens incrementally as they are generated.

SSE Event Types

Event	When Fired	Key Data
<code>message_start</code>	Stream begins	Message object with <code>id</code> , <code>model</code> , <code>usage.input_tokens</code>
<code>content_block_start</code>	New content block begins	Block <code>index</code> and <code>type</code> (text, tool_use, thinking)
<code>content_block_delta</code>	Incremental content	<code>text_delta</code> , <code>input_json_delta</code> , or <code>thinking_delta</code>
<code>content_block_stop</code>	Block is complete	Block <code>index</code>
<code>message_delta</code>	Message-level update	<code>stop_reason</code> , <code>usage.output_tokens</code>
<code>message_stop</code>	Stream ends	Empty — signals completion
<code>ping</code>	Keep-alive	Empty — prevents timeout
<code>error</code>	Stream error	Error <code>type</code> and <code>message</code>

Python Streaming Example

```
from anthropic import Anthropic client = Anthropic() # Context manager approach - recommended
with client.messages.stream( model="claude-opus-5", max_tokens=4096, messages=[{"role": "user",
"content": "Explain async/await in Python."}] ) as stream:
    for text in stream.text_stream:
        print(text, end="", flush=True) # Get the final assembled message
    final_message = stream.get_final_message()
    print(f"\nTokens used: {final_message.usage.output_tokens}")
```

TypeScript Streaming Example

```
import Anthropic from "@anthropic-ai/sdk"; const client = new Anthropic(); const stream = client.messages.stream({ model: "claude-opus-5", max_tokens: 4096, messages: [{ role: "user", content: "Explain async/await." }], }); stream.on("text", (text) => process.stdout.write(text)); const message = await stream.finalMessage(); console.log(`\nTokens: ${message.usage.output_tokens}`);
```

TYPESCRIPT

Streaming with Tool Use

When Claude makes a tool call during streaming, the events arrive as:

1. `content_block_start` with `type: "tool_use"` containing the tool `name` and `id`
2. Multiple `content_block_delta` events with `input_json_delta` (partial JSON for tool arguments)
3. `content_block_stop` when the tool call arguments are complete
4. `message_delta` with `stop_reason: "tool_use"`

Eager Input Streaming (Fine-Grained Tool Streaming)

Set `eager_input_streaming: true` on a tool definition to receive large tool inputs as they are generated, rather than waiting for the server to buffer the complete input. This is **not a beta feature**—no beta header needed. Use it only on streaming requests with client-side tools. The client must validate the parsed input against the schema, since the SDK's tolerant parser may silently return a truncated result.

PART 04

Tool Use & Function Calling — Production Patterns

Tool Definition Schema

```
{ "name": "get_weather", "description": "Get current weather for a city. Returns temperature in Fahrenheit.", "input_schema": { "type": "object", "properties": { "city": { "type": "string", "description": "City name, e.g. 'San Francisco' }, "units": { "type": "string", "enum": ["fahrenheit", "celsius"] } }, "required": ["city"], "additionalProperties": false }, "strict": true }
```

Complete Agentic Loop

```
import json from anthropic import Anthropic client = Anthropic() tools = [{ # tool definitions as above }] def execute_tool(name: str, inputs: dict) -> str: if name == "get_weather": return json.dumps({"temp": 72, "condition": "sunny"}) return json.dumps({"error": "Unknown tool"}) def run_agent(user_message: str): messages = [{"role": "user", "content": user_message}] while True: response = client.messages.create( model="claude-opus-5", max_tokens=4096, tools=tools, messages=messages ) # Append assistant response (full content, not just text) messages.append({"role": "assistant", "content": response.content}) if response.stop_reason == "end_turn": return response.content[-1].text # Process tool calls tool_results = [] for block in response.content: if block.type == "tool_use": result = execute_tool(block.name, block.input) tool_results.append({ "type": "tool_result", "tool_use_id": block.id, "content": result }) messages.append({"role": "user", "content": tool_results})
```

tool_choice Modes

Mode	Behavior	Notes
<code>{"type": "auto"}</code>	Claude decides whether to call a tool	Default. Works on all models.
<code>{"type": "any"}</code>	Force Claude to call some tool	Removed on Fable 5.1 / Opus 5.5 (returns 400).
<code>{"type": "tool", "name": "X"}</code>	Force a specific tool	Removed on Fable 5.1 / Opus 5.5 (returns 400).
<code>{"type": "none"}</code>	Disable tool calling	Works on all models.

Use `disable_parallel_tool_use: true` inside `tool_choice` to limit Claude to one tool call per turn.

Strict Tool Use

Set `strict: true` as a **top-level field on the tool definition** (alongside `name` , `description` , `input_schema`), not on `tool_choice` . The schema must include `additionalProperties: false` and list all properties in `required` . This guarantees the tool call input validates exactly against your schema.

Error Handling in Tool Results

```
# Return errors using is_error field - never drop a tool_result { "type": "tool_result",  
"tool_use_id": block.id, "content": "Error: City 'Atlantis' not found in database.", "is_error":  
True }  
PYTHON
```

Server-Side Tools

These run on Anthropic's infrastructure. No client-side execution needed—results arrive as content blocks in the response.

Tool	Type String	Result Block
Web Search	<code>web_search_20260209</code>	<code>web_search_tool_result</code>
Web Fetch	<code>web_fetch_20260209</code>	<code>web_fetch_tool_result</code>
Code Execution	<code>code_execution_20260521</code>	<code>bash_code_execution_tool_result</code>

Do not combine web search + code execution

The `web_search_20260209` and `web_fetch_20260209` tools have built-in dynamic filtering that uses code execution under the hood. Separately declaring `code_execution` alongside them creates a confusing second execution environment.

Client-Side Anthropic Tools

These are pre-defined by Anthropic but executed on your infrastructure. They use special `type` strings and have no `input_schema` .

Tool	Type String	Purpose
Bash	<code>bash_20250124</code>	Execute shell commands
Text Editor	<code>text_editor_20250728</code>	Read, write, and edit files
Computer Use	<code>computer_20251124</code>	GUI interaction (screenshots, clicks, keyboard)
Memory	<code>memory_20250818</code>	Persistent memory across conversations

Tool Use Token Overhead

Each tool definition adds tokens to the system prompt. Approximate overhead per model:

- A simple tool definition adds roughly 200–400 input tokens per tool.
- For large tool sets (50+ tools), use **tool search** (`tool_search_tool_regex_20251119` or `tool_search_tool_bm25_20251119`) with `defer_loading: true` on most tools to reduce per-request token overhead.

PART 05

Structured Outputs & JSON Mode

output_config.format with json_schema

Constrains the response to match a JSON Schema. Use `output_config: {format: {type:`

`"json_schema", json_schema: {...}}}` on `messages.create()` .

Breaking Change: output_format is deprecated

The old `output_format` parameter is deprecated. Use `output_config: {format: {...}}` instead. This is a required migration for all new code.

Python Example with Pydantic

```
from pydantic import BaseModel from anthropic import Anthropic client = Anthropic() class PYTHON
MovieReview(BaseModel): title: str rating: float summary: str pros: list[str] cons: list[str] #
parse() auto-extracts the JSON schema from the Pydantic model response = client.messages.parse(
model="claude-opus-5", max_tokens=1024, messages=[{"role": "user", "content": "Review the movie
Inception"}], output_type=MovieReview ) # response.parsed is a validated MovieReview instance
review = response.parsed print(f"{review.title}: {review.rating}/10")
```

TypeScript Example with Zod

```
import Anthropic from "@anthropic-ai/sdk"; import { zodOutputFormat } from "@anthropic- TYPESCRIPT
ai/sdk/helpers/zod"; import { z } from "zod"; const client = new Anthropic(); const MovieReview
= z.object({ title: z.string(), rating: z.number(), summary: z.string(), pros:
z.array(z.string()), cons: z.array(z.string()), }); const response = await
client.messages.create({ model: "claude-opus-5", max_tokens: 1024, messages: [{ role: "user",
content: "Review the movie Inception" }], output_config: zodOutputFormat(MovieReview, {});
```

Strict Tool Use as Alternative

For simpler structured data, you can define a tool with `strict: true` and instruct Claude to call it. This guarantees schema conformance on the tool input. However, `output_config.format` is preferred for general structured output because it constrains the *response* directly without requiring a fake tool call.

Schema Limitations

- No recursive schemas (no `$ref` to self)
- No `minLength` / `maxLength` on strings

- No `minimum` / `maximum` on numbers
- No `pattern` (regex) constraints
- `additionalProperties: false` is **required**
- All properties must be listed in `required`
- Incompatible with **citations** (`citations: {enabled: true}` returns a 400)

Performance Notes

- **Grammar compilation** — The first request with a new schema incurs a compilation cost. Subsequent requests with the same schema use a **24-hour cache**.
- **Cache key** — The full JSON schema is the cache key. Any change (even whitespace) creates a new compilation.

PART 06

Prompt Caching — Implementation Guide

Two Caching Modes

Automatic Caching

Top-level `cache_control: {"type": "ephemeral"}` on the `messages.create()` call. The API automatically determines optimal breakpoints. Simplest option.

Explicit Breakpoints

Per-block `cache_control` on individual content blocks within `system`, `tools`, or `messages`. Max 4 breakpoints per request. Gives fine-grained control.

Python Code Examples

```
response = client.messages.create( model="claude-opus-5", max_tokens=4096, cache_control={"type": "ephemeral"}, # top-level auto-caching system="You are a helpful assistant with expertise in Python.", messages=[{"role": "user", "content": "Explain decorators."}] )
```

PYTHON — AUTOMATIC

```
response = client.messages.create( model="claude-opus-5", max_tokens=4096, system=[{"type": "text", "text": "<very long system prompt with guidelines...>", "cache_control": {"type": "ephemeral"} # breakpoint } ], messages=[{"role": "user", "content": "Summarize the guidelines."}] )
```

PYTHON — EXPLICIT BREAKPOINTS

TTL Options

TTL	Write Cost	Default?	Configuration
5 minutes	1.25x standard input price	Yes (default)	<code>{"type": "ephemeral"}</code>
1 hour	2x standard input price	No	<code>{"type": "ephemeral", "ttl": "1h"}</code>

Cache Read Pricing

Model	Cache Read Rate
Most models (Opus 5, Sonnet 5, etc.)	0.1x standard input price
Fable 5.1	0.025x standard input price

Model	Cache Read Rate
Opus 5.5	0.05x standard input price

Minimum Token Requirements

The prefix must meet a minimum token count or it silently will not cache:

- 512 tokens — Opus 5, Sonnet 5, Fable 5, Fable 5.1
- 1024 tokens — Haiku 4.5
- 2048–4096 tokens — Some older models

Tracking Cache Performance

```
usage = response.usage print(f"New input tokens: {usage.input_tokens}") print(f"Cache write (new): {usage.cache_creation_input_tokens}") print(f"Cache reads (hits): {usage.cache_read_input_tokens}") # If cache_read_input_tokens is 0 across requests, check for invalidators
```

PYTHON

Cache Pre-Warming

Send a request with `max_tokens: 0` to populate the cache without generating output. Useful for warming the cache before a burst of traffic.

Cache Invalidation Hierarchy

Prompt caching uses **prefix matching**. The render order is:

tools → system → messages

Any byte change anywhere in the prefix invalidates everything after it.

- ↓ Keep stable content first
- ↓ Put volatile content (timestamps, IDs) after the last breakpoint
- ↓ Use deterministic JSON serialization for tool definitions

Common Silent Invalidators

`datetime.now()` in system prompts, unsorted JSON keys in tool schemas, varying tool sets between requests, per-request UUIDs before the cache breakpoint. If `cache_read_input_tokens` is consistently 0, audit your prefix for any of these.

Multi-Turn Automatic Caching

In multi-turn conversations, the growing message history naturally extends the cached prefix. Each new turn adds content after the previous cache breakpoint, so prior turns hit cache while only the new content incurs write costs.

Key Rule: Cache Reads and Rate Limits

Cache read tokens do **NOT** count toward ITPM (Input Tokens Per Minute) rate limits. This means heavily cached workloads can process far more total tokens per minute than uncached ones.

PART 07

Batch Processing

When to Use Batches

- **Large-volume processing** — Thousands of requests that do not need real-time responses.
- **Evaluations** — Running test suites against prompts.
- **Data processing** — Classification, extraction, or summarization of large datasets.
- **Cost optimization** — 50% cheaper than standard API calls.

JSONL Request Format

Each line is an independent request with a `custom_id` for correlation:

```
{
  "custom_id": "req-001",
  "params": {
    "model": "claude-sonnet-5",
    "max_tokens": 1024,
    "messages": [
      {
        "role": "user",
        "content": "Classify: positive or negative"
      }
    ]
  }
}
{"custom_id": "req-002",
 "params": {
  "model": "claude-sonnet-5",
  "max_tokens": 1024,
  "messages": [
    {
      "role": "user",
      "content": "Summarize this review..."
    }
  ]
}
```

Creating a Batch (Python)

```
from anthropic import Anthropic
from anthropic.types.messages import MessageCreateParamsNonStreaming
from anthropic.types.messages.batch_create_params import Request
client = Anthropic()
batch = client.messages.batches.create(
    requests=[
        Request(
            custom_id="sentiment-001",
            params=MessageCreateParamsNonStreaming(
                model="claude-sonnet-5",
                max_tokens=256,
                messages=[{"role": "user", "content": "Classify as positive/negative: 'Great product!'"}]
            ),
        ),
        Request(
            custom_id="sentiment-002",
            params=MessageCreateParamsNonStreaming(
                model="claude-sonnet-5",
                max_tokens=256,
                messages=[{"role": "user", "content": "Classify as positive/negative: 'Terrible experience.'"}]
            ),
        ),
    ]
)
print(f"Batch ID: {batch.id}")
```

Polling for Results

```
import time
while True:
    batch = client.messages.batches.retrieve(batch.id)
    print(f"Status: {batch.processing_status}")
    if batch.processing_status == "ended":
        break
    time.sleep(60) # poll every minute
```

Retrieving Results

```
for result in client.messages.batches.results(batch.id): match result.result.type: case
"succeeded": message = result.result.message print(f"{result.custom_id}:
{message.content[0].text}") case "errored": print(f"{result.custom_id}: ERROR -
{result.result.error}") case "canceled": print(f"{result.custom_id}: Canceled") case "expired":
print(f"{result.custom_id}: Expired (24h limit)")
```

Batch Processing Key Facts

- **50% cost savings** compared to standard API requests
- **100,000 requests** maximum per batch
- **24-hour processing window** — results expire after 24 hours
- Results arrive in **any order** — key by `custom_id`, never by position
- Processing statuses: `in_progress`, `ended`, `canceled`, `expired`
- Result types: `succeeded`, `errored`, `canceled`, `expired`

PART 08

Thinking & Reasoning — Developer Guide

Adaptive Thinking (Current Standard)

On all current-generation models (Opus 4.6+, Sonnet 4.6+, Sonnet 5, Fable 5/5.1), use adaptive thinking. Claude dynamically decides when and how deeply to reason.

```
response = client.messages.create( model="claude-opus-5", max_tokens=16000, thinking={"type": "adaptive", "display": "summarized"}, messages=[{"role": "user", "content": "Prove that sqrt(2) is irrational."}] )
```

Extended Thinking (Legacy)

Only supported on Haiku 4.5 and older models. Uses a fixed token budget. This is deprecated for new code.

```
response = client.messages.create( model="claude-haiku-4-5", max_tokens=16000, thinking={"type": "enabled", "budget_tokens": 8000}, messages=[{"role": "user", "content": "Solve this math problem..."}] ) # budget_tokens must be less than max_tokens, minimum 1024
```

Effort Parameter

Controls thinking depth. Set inside `output_config` , not top-level:

```
response = client.messages.create( model="claude-opus-5", max_tokens=4096, thinking={"type": "adaptive"}, output_config={"effort": "xhigh"}, messages=[...] )
```

Level	Use Case	Token Spend
low	Simple tasks, subagents, classification	Minimal
medium	Routine work, cost-sensitive routes	Low
high	Default for most models. Balanced quality/cost.	Moderate
xhigh	Coding, agentic tasks. Best for most dev work.	High
max	When correctness matters more than cost.	Very High

Thinking Blocks in Responses

When thinking is active, the response `content` array may include thinking blocks before text blocks:

```
[ { "type": "thinking", "thinking": "Let me analyze this step by step...", "signature": "dGhpbmtbmdfc2lnbmF0dXJl...", "display": "omitted" }, { "type": "text", "text": "The answer is 42." } ]
```

RESPONSE CONTENT ARRAY

Multi-Turn with Thinking (Signature Required)

When continuing a conversation, you **must** pass thinking blocks back unchanged, including their `signature` field. Never modify the thinking text or signature.

```
# Append the FULL content (including thinking blocks) back messages.append({"role": "assistant", "content": response.content})
messages.append({"role": "user", "content": "Can you elaborate?"})
# Thinking blocks with matching signatures will be accepted # Thinking blocks from a different model are silently dropped
```

PYTHON

Thinking with Tool Use

Thinking blocks can appear between tool calls. In an agentic loop, Claude may think before each tool call and again after processing results. The content array can look like: `[thinking, tool_use, tool_use, ...]` on one turn, then after tool results: `[thinking, text]` on the next.

Token Billing

- Thinking tokens count as **output tokens** and are billed at the output rate.
- They count toward `max_tokens`.
- With `display: "omitted"` (default on newer models), thinking blocks are still generated and billed, but the `thinking` field is empty.
- With `display: "summarized"`, you get a readable summary (still not the raw chain of thought).

When to Use Thinking

Thinking significantly improves performance on: mathematical proofs, complex coding tasks, multi-step analysis, agentic decision-making, and tasks requiring careful reasoning. For simple classification or extraction, lower effort or no thinking may be more cost-effective.

PART 09

Rate Limits, Error Handling & Production Resilience

Three Rate Limit Types

Limit Type	Abbreviation	What It Measures
Requests Per Minute	RPM	Total API requests per minute
Input Tokens Per Minute	ITPM	Input tokens processed per minute
Output Tokens Per Minute	OTPM	Output tokens generated per minute

Usage Tiers

Tier	Monthly Cap	Access Level
Start	\$500/month	Initial tier for new accounts
Build	\$1,000/month	Increased limits after credit purchase
Scale	\$200,000/month	Production workloads, highest standard limits
Custom	Negotiated	Enterprise, custom rate limits

Scale Tier Rate Limits (Example)

Model	RPM	ITPM	OTPM
Claude Opus 5.5	10,000	10,000,000	2,000,000
Claude Opus 5	10,000	10,000,000	2,000,000
Claude Sonnet 5	10,000	10,000,000	2,000,000
Claude Haiku 4.5	10,000	10,000,000	2,000,000

Cache-Aware ITPM

Cached tokens (reported as `cache_read_input_tokens`) do **NOT** count toward the ITPM rate limit. Only `input_tokens` and `cache_creation_input_tokens` count. This means a well-cached workload can effectively process far more total input per minute.

Token Bucket Algorithm

Rate limits use a **token bucket** model. The bucket fills at a constant rate (the limit) and drains with each request. Short bursts above the per-minute limit are allowed if the bucket has capacity, but sustained over-limit usage will trigger 429s.

HTTP 429 Handling

```
from anthropic import Anthropic, RateLimitError client = Anthropic() try: response = client.messages.create(...) except RateLimitError as e: # SDK automatically retries 429s with exponential backoff # This fires only after all retries are exhausted retry_after = e.response.headers.get("retry-after") print(f"Rate limited. Retry after: {retry_after}s")
```

PYTHON

Response Headers

Header	Description
anthropic-ratelimit-requests-limit	Max RPM for your tier
anthropic-ratelimit-requests-remaining	Remaining RPM in current window
anthropic-ratelimit-requests-reset	When RPM bucket refills (ISO 8601)
anthropic-ratelimit-input-tokens-limit	Max ITPM
anthropic-ratelimit-input-tokens-remaining	Remaining ITPM
anthropic-ratelimit-output-tokens-limit	Max OTPM
anthropic-ratelimit-output-tokens-remaining	Remaining OTPM

Spend Limit Errors

Distinct from rate limits: `error_code: "enforced_spend_limit_reached"` means you have hit your monthly spending cap. This is a 400-level error, not a 429. Solution: increase the spend limit in the Console, not retry.

SDK Automatic Retries

Both the Python and TypeScript SDKs automatically retry on 429 (rate limit), 408 (timeout), 409 (conflict), and 5xx (server error) with exponential backoff. Default: **2 retries** (configurable via `max_retries`). Wall-clock time can reach `timeout x (max_retries + 1)` .

Common HTTP Error Codes

Code	Type	Meaning	Action
400	<code>invalid_request_error</code>	Malformed request, invalid params	Fix the request — do not retry
401	<code>authentication_error</code>	Invalid or missing API key	Check ANTHROPIC_API_KEY
403	<code>permission_error</code>	API key lacks required scope	Check key permissions
404	<code>not_found_error</code>	Resource not found	Check model ID / endpoint
429	<code>rate_limit_error</code>	Rate limit exceeded	Retry with backoff (SDK auto-retries)
500	<code>api_error</code>	Internal server error	Retry with backoff
529	<code>overloaded_error</code>	API is temporarily overloaded	Retry with backoff

Best Practice: Catch a Chain, Not One Broad Class

Use specific exception types in order of specificity: `NotFoundError` → `RateLimitError` → `APIStatusError` → `APIConnectionError` . A single broad `except APIStatusError` loses the distinction between retryable and non-retryable failures.

PART 10

Testing, Evaluation & Quality Assurance

Defining SMART Success Criteria

Before building evaluations, define **SMART** criteria: Specific, Measurable, Achievable, Relevant, Time-bound. Example: "The model should correctly extract all invoice line items with at least 95% accuracy on the test set."

Five Evaluation Types

1. Exact Match

```
def eval_exact_match(expected: str, actual: str) → float: return 1.0 if
expected.strip().lower() == actual.strip().lower() else 0.0
```

PYTHON

2. Cosine Similarity (SBERT)

```
from sentence_transformers import SentenceTransformer from sklearn.metrics.pairwise import
cosine_similarity model = SentenceTransformer("all-MiniLM-L6-v2") def
eval_cosine_similarity(expected: str, actual: str) → float: embeddings =
model.encode([expected, actual]) score = cosine_similarity([embeddings[0]], [embeddings[1]])[0]
[0] return float(score)
```

PYTHON

3. ROUGE-L (Longest Common Subsequence)

```
from rouge_score import rouge_scorer scorer = rouge_scorer.RougeScorer(["rougeL"],
use_stemmer=True) def eval_rouge_l(expected: str, actual: str) → float: scores =
scorer.score(expected, actual) return scores["rougeL"].fmeasure
```

PYTHON

4. LLM-Based Likert Scale

```
def eval_llm_likert(question: str, answer: str, reference: str) → int: # Use a DIFFERENT ...
than the one being evaluated response = client.messages.create( model="claude-sonnet-5",
max_tokens=100, system="Rate the answer on a 1-5 scale for accuracy and completeness.",
messages=[{"role": "user", "content": f"Question: {question}\nReference: {reference}\nAnswer:
{answer}\nScore (1-5):"}] ) return int(response.content[0].text.strip())
```

PYTHON

5. LLM-Based Binary Classification

```
def eval_llm_binary(question: str, answer: str, criteria: str) → bool: response =  
client.messages.create( model="claude-sonnet-5", max_tokens=10, system="You are a strict  
evaluator. Answer PASS or FAIL only.", messages=[{"role": "user", "content": f"Criteria:  
{criteria}\nQuestion: {question}\nAnswer: {answer}\nVerdict:"}] ) return  
response.content[0].text.strip().upper() == "PASS"
```

Best Practices

- Use a **different model** for evaluation than the one being tested (prevents self-bias).
- Include **edge cases**: empty inputs, very long inputs, adversarial prompts.
- **Automate** your eval suite—run it after every prompt change.
- **Combine metrics**: use both automated (exact match, ROUGE) and LLM-based judges.
- Track scores over time to detect regressions.

Prompt Engineering Iteration Loop

1. **Define success criteria** (SMART, measurable)
↓
 2. **Build evaluation set** (20-50 test cases minimum)
↓
 3. **Draft initial prompt**
↓
 4. **Run evaluations** (automated + manual review)
↓
 5. **Analyze failures** (categorize error types)
↓
 6. **Refine prompt** (address specific failure modes)
↓
- Loop back to step 4 until criteria are met

PART 11

Security, Data Policies & Production Deployment

API Key Management

- Store keys in environment variables (`ANTHROPIC_API_KEY`) or a secrets manager.
- **Never** embed keys in client-side code, Git repos, or log files.
- Set **key expiration** dates and rotate regularly.
- Use **Workspaces** for team isolation—each workspace can have its own keys and rate limits.
- OAuth-based authentication (`ant auth login`) is preferred over long-lived API keys for developer workflows.

Zero Data Retention (ZDR)

Feature	ZDR-Eligible?	Notes
Messages API	Yes	Standard API requests, no data stored
Batch API	Yes	Requests processed and discarded
Prompt Caching	Yes	Cached data is ephemeral, not persisted
Fable 5 / Fable 5.1	Conditional	Requires 30-day data retention unless expressly authorized
Managed Agents	No	Session state requires data retention
Claude.ai (consumer)	No	Conversations stored for product improvement

HIPAA Readiness

Anthropic offers HIPAA-compliant deployments through the API with a Business Associate Agreement (BAA). Requirements: ZDR-eligible features only, no PHI in prompt caching with extended TTLs, proper access controls.

Data Processor Roles

Platform	Role
Claude API (Anthropic)	Data processor (your org is controller)
Amazon Bedrock	AWS is processor (Anthropic is subprocessor)

Platform	Role
Google Vertex AI	Google is processor (Anthropic is subprocessor)
claude.ai (consumer)	Anthropic is controller

Input Validation & Content Filtering

- Validate and sanitize all user inputs before sending to the API.
- Set `metadata.user_id` for per-user abuse tracking.
- Implement content moderation on both inputs and outputs for user-facing applications.
- Use **web search/fetch domain filtering** (`allowed_domains` / `blocked_domains`) to restrict tool access.

Workspaces for Team Isolation

- Each workspace has **independent API keys** and rate limits.
- Usage is tracked and billed per workspace.
- Members can be assigned to specific workspaces with role-based access.
- Useful for separating production / staging / development environments.

Cost Optimization Strategies

Model Selection

Use the cheapest model that meets quality requirements. Haiku 4.5 for classification, Sonnet 5 for balanced tasks, Opus 5 for complex reasoning.

Prompt Caching

Cache stable prefixes (system prompts, tool definitions). Cache reads cost 0.1x. Biggest single lever for cost reduction.

Batch Processing

50% cost savings for non-latency-sensitive workloads. Use for evaluations, data processing, bulk operations.

Effort Tuning

Lower effort on simple tasks. A single model at lower effort often matches a cheaper model at high effort, while keeping cache benefits.

Monitoring

- **Usage page** in the Console — total tokens, cost breakdown by model and workspace.
- **Rate limit charts** — track utilization vs. limits across RPM, ITPM, OTPM.
- **Cache hit rates** — monitor `cache_read_input_tokens` vs. `cache_creation_input_tokens` .

- **Response usage** — every API response includes `usage` for per-request tracking.

PART 12

Practice Test — 50 Developer MCQs

Instructions

Each question has exactly one correct answer, highlighted in green. Read the explanation for each question to understand the reasoning behind the correct answer. Questions cover developer-focused topics: API patterns, SDK usage, tool use, structured outputs, caching, batching, thinking, rate limits, error handling, and production deployment.

Q1 API & SDK

Which two parameters are required on every `POST /v1/messages` request in addition to `messages` ?

- A `model` and `system`
- B `model` and `max_tokens`**
- C `model` and `temperature`
- D `max_tokens` and `stream`

Correct: B. The Messages API requires `model` , `max_tokens` , and `messages` . All other parameters (`system` , `temperature` , `stream` , etc.) are optional.

Q2 API & SDK

In a multi-turn conversation, after receiving a response from the API, what should you append to the `messages` array?

- A Just the text string from `response.content[0].text`
- B The full `response.content` array as the assistant message content**
- C The entire `response` object
- D A new `{"role": "assistant", "text": response.content[0].text}` object

Correct: B. You must append the full `response.content` (not just text) to preserve thinking blocks, tool_use blocks, and their signatures. Extracting only the text string silently loses compaction state and thinking signatures.

Q3 API & SDK

Which SSE event type delivers the final `stop_reason` and total `output_tokens` during streaming?

- A `message_stop`
- B `content_block_stop`
- C `message_delta`**
- D `message_start`

Correct: C. The `message_delta` event carries `stop_reason` and `usage.output_tokens`. `message_stop` is an empty signal that the stream has ended. `message_start` carries `input_tokens`.

Q4 API & SDK

In the Python SDK, which is the recommended way to stream responses and get the assembled final message?

- A `client.messages.create(stream=True)` and manually parse SSE events
- B `with client.messages.stream(...) as stream:` context manager with `stream.get_final_message()`**
- C `client.messages.stream_text(...)`
- D `async for chunk in client.messages.create(stream=True):`

Correct: B. The SDK provides the `client.messages.stream()` context manager with `text_stream` for incremental text and `get_final_message()` for the assembled message. This is preferred over manually parsing SSE events.

Q5 API & SDK

What is the maximum `max_tokens` value supported by Claude Opus 5 and other current-generation models?

- A 4,096
- B 16,384
- C 65,536
- D 128,000**

Correct: D. Current-generation models (Opus 4.6+, Sonnet 4.6+, Sonnet 5, Fable 5/5.1) support up to 128K `max_tokens`. However, values above ~16K require streaming to avoid HTTP timeouts.

Q6 API & SDK

Where should sampling parameters (`temperature` , `top_p` , `top_k`) NOT be used?

- A** Claude Haiku 4.5
- B** Claude Sonnet 4.6
- C** Claude Opus 5 (or any 4.7+ / Sonnet 5 model)
- D** All Claude models support these parameters

Correct: C. Sampling parameters (`temperature` , `top_p` , `top_k`) are removed on Opus 4.7+, Opus 4.8, Opus 5, Sonnet 5, and Fable 5/5.1. Sending them returns a 400 error. They are still allowed on Opus 4.6, Sonnet 4.6, and Haiku 4.5.

Q7 API & SDK

What does a `stop_reason` of "refusal" indicate?

- A** The API key does not have sufficient permissions
- B** The request exceeded the rate limit
- C** A safety classifier declined the request (HTTP 200, check `stop_details.category`)
- D** The model encountered an internal error

Correct: C. A "refusal" stop reason is returned as HTTP 200 (not an error status code). The `stop_details` object contains a `category` field (e.g., "cyber", "bio") and an `explanation` . Always check `stop_reason` before reading `content` .

Q8 API & SDK

What is the default number of automatic retries in the Python and TypeScript SDKs?

- A** 0 (no retries)
- B** 1
- C** 2
- D** 5

Correct: C. Both SDKs default to 2 retries with exponential backoff for status codes 408, 409, 429, and 5xx, plus connection errors. Configurable via `max_retries` .

Q9 TOOL USE

Where should the `strict: true` field be placed in a tool definition?

- A Inside `tool_choice`
- B Inside `input_schema`
- C As a top-level field on the tool definition, alongside `name` and `description`
- D As a request-level parameter on `messages.create()`

Correct: C. `strict: true` is a top-level field on the tool definition object (alongside `name`, `description`, `input_schema`). The schema must also include `additionalProperties: false` and all properties in `required`.

Q10 TOOL USE

In the agentic loop, after Claude returns a `tool_use` stop reason, what is the correct message structure to send back tool results?

- A A new assistant message with the tool result text
- B A user message containing an array of `tool_result` content blocks, each referencing `tool_use_id`
- C A system message with the tool results
- D An assistant message with a `tool_result` block

Correct: B. Tool results are sent as a `user` message containing `tool_result` blocks. Each block must reference the `tool_use_id` from the corresponding `tool_use` block. All parallel tool results go in a single user message.

Q11 TOOL USE

What is the correct type string for the Anthropic-defined bash tool?

- A `"bash"`
- B `"bash_20250124"`
- C `"shell_tool"`
- D `"computer_bash"`

Correct: B. Anthropic-defined tools use versioned type strings: `bash_20250124` for bash, `text_editor_20250728` for the text editor, `computer_20251124` for computer use, and `memory_20250818` for memory. They have no `input_schema`.

Q12 TOOL USE

When Claude makes multiple parallel tool calls in a single response, how should you return the results?

- A Send each tool result in a separate user message
- B Send all tool results in a single user message containing all `tool_result` blocks**
- C Send each tool result in a separate assistant message
- D Ignore the parallel calls and process only the first one

Correct: B. All parallel tool results must go in a single user message. Splitting them across multiple messages silently trains Claude to stop making parallel calls, degrading performance.

Q13 TOOL USE

Which `tool_choice` type is removed on Claude Fable 5.1 and returns a 400 error?

- A `{"type": "auto"}`
- B `{"type": "none"}`
- C `{"type": "any"}` and `{"type": "tool", "name": "..."}`**
- D All `tool_choice` types are supported on all models

Correct: C. Forced tool use (`"any"` and `"tool"`) is removed on Fable 5.1 and Opus 5.5. Use `"auto"` with a prompt instruction instead, and `strict: true` for schema-valid arguments.

Q14 TOOL USE

What is the current type string for the server-side web search tool?

- A `web_search`
- B `web_search_20250305`
- C `web_search_20260209`**
- D `server_web_search`

Correct: C. The current web search tool type is `web_search_20260209` , which includes dynamic filtering. The older `web_search_20250305` is the basic variant for models older than Opus 4.6.

Q15 TOOL USE

How should a tool execution error be reported back to the API?

- A Omit the `tool_result` block entirely
- B Throw an exception in the tool handler
- C Return a `tool_result` block with `is_error: true` and an error message in `content`**
- D Return an empty `tool_result` block

Correct: C. Always return a `tool_result` with `is_error: true` and a descriptive error message. Never drop a `tool_result` block — Claude expects a result for every `tool_use` it made.

Q16 TOOL USE

Why should you NOT declare `code_execution` separately when using `web_search_20260209` ?

- A Code execution is not compatible with web search
- B It causes a rate limit error
- C The `_20260209` variants have built-in code execution for dynamic filtering; a second environment confuses the model**
- D Code execution costs extra tokens when combined with search

Correct: C. The `web_search_20260209` and `web_fetch_20260209` tools use code execution under the hood for dynamic filtering. Declaring a separate `code_execution` tool creates a confusing second execution environment.

Q17 STRUCTURED OUTPUTS

What is the correct parameter path for specifying a JSON schema output format?

- A `output_format: {type: "json_schema", ...}`
- B `output_config: {format: {type: "json_schema", json_schema: {...}}}`**
- C `response_format: {type: "json_object"}`
- D `format: {type: "json_schema", ...}`

Correct: B. Structured outputs use `output_config.format` with `type: "json_schema"` and the schema in `json_schema`. The old `output_format` parameter is deprecated.

Q18 STRUCTURED OUTPUTS

In the Python SDK, which method auto-validates the response against a Pydantic model?

- A `client.messages.create()`
- B `client.messages.parse()`**
- C `client.messages.validate()`
- D `client.messages.structured()`

Correct: B. `client.messages.parse()` accepts an `output_type` parameter with a Pydantic BaseModel class, automatically extracts the JSON schema, and returns a `response.parsed` attribute with the validated instance.

Q19 STRUCTURED OUTPUTS

In TypeScript, which helper function converts a Zod schema to the `output_config` format?

- A `zodOutputFormat()`**
- B `zodToJsonSchema()`
- C `zodResponseFormat()`
- D `outputSchema()`

Correct: A. `zodOutputFormat()` from `@anthropic-ai/sdk/helpers/zod` converts a Zod schema to the `output_config` object expected by `messages.create()`.

Q20 STRUCTURED OUTPUTS

Which JSON Schema constraints are NOT supported in Claude's structured output schemas?

- A `type` , `properties` , `enum`
- B `minLength` , `maxLength` , `pattern` , `recursive` , `$ref`**
- C `required` , `additionalProperties`
- D `array` , `object` , `string`

Correct: B. Claude's structured outputs do not support `minLength` , `maxLength` , `minimum` , `maximum` , `pattern` (regex), or recursive schemas (`$ref` to self). `additionalProperties: false` is required, not optional.

Q21 STRUCTURED OUTPUTS

How long does the grammar compilation cache last for a given JSON schema?

- A 5 minutes
- B 1 hour
- C 24 hours**
- D 7 days

Correct: C. The first request with a new schema incurs a compilation cost. The compiled grammar is cached for 24 hours. Any change to the schema (even whitespace) creates a new compilation.

Q22 STRUCTURED OUTPUTS

What happens if you use `output_config.format` with `citations: {enabled: true}` on a document block?

- A Both features work together seamlessly
- B Citations are silently ignored
- C The API returns a 400 error (they are incompatible)**
- D The structured output format includes citation metadata

Correct: C. Structured outputs (`output_config.format`) and citations are incompatible features. Using both returns a 400 error. You must choose one or the other.

Q23 PROMPT CACHING

What is the difference between automatic and explicit prompt caching?

- A Automatic uses a different API endpoint; explicit uses standard `messages.create()`
- B Automatic uses `cache_control` with auto breakpoints; explicit `cache_control` on individual top-level places per-block content blocks**
- C Automatic is free; explicit has a 1.25x write cost
- D Automatic caches responses; explicit caches inputs

Correct: B. Automatic caching sets `cache_control: {"type": "ephemeral"}` at the top level and the API picks optimal breakpoints. Explicit caching places `cache_control` on individual blocks within system/tools/messages for fine-grained control (max 4 breakpoints).

Q24 PROMPT CACHING

What is the write cost multiplier for the 1-hour TTL cache option?

- A** 1.0x (same as standard)
- B** 1.25x
- C** 2.0x
- D** 3.0x

Correct: C. The 1-hour TTL (`"ttl": "1h"`) has a 2x write cost. The default 5-minute TTL has a 1.25x write cost. Cache reads are 0.1x for most models regardless of TTL.

Q25 PROMPT CACHING

What is the minimum cacheable prefix size for Claude Opus 5?

- A** 128 tokens
- B** 256 tokens
- C** 512 tokens
- D** 2048 tokens

Correct: C. The minimum cacheable prefix is 512 tokens for Opus 5, Sonnet 5, Fable 5, and Fable 5.1. For Haiku 4.5, it is 1024 tokens. Prefixes shorter than the minimum silently will not cache.

Q26 PROMPT CACHING

What is the correct render order used for cache prefix matching?

- A** `messages` → `system` → `tools`
- B** `system` → `tools` → `messages`
- C** `tools` → `system` → `messages`
- D** `system` → `messages` → `tools`

Correct: C. The cache prefix render order is `tools` → `system` → `messages` . Any byte change anywhere in the prefix invalidates everything after it. Keep stable content (frozen system prompt, deterministic tool list) first.

Q27 PROMPT CACHING

How do you pre-warm a cache without generating any output?

- A Set `stream: false` and `temperature: 0`
- B Send a request with `max_tokens: 0`**
- C Use a dedicated `/v1/cache/warm` endpoint
- D Set `cache_control: {"type": "pre_warm"}`

Correct: B. Sending a request with `max_tokens: 0` populates the cache without generating any output. This is useful for warming the cache before a burst of traffic.

Q28 PROMPT CACHING

Do cache read tokens count toward the ITPM (Input Tokens Per Minute) rate limit?

- A Yes, all input tokens count equally
- B No, cache read tokens are excluded from ITPM**
- C Only on the Start and Build tiers
- D Only with the 1-hour TTL

Correct: B. Cache read tokens (`cache_read_input_tokens`) do NOT count toward the ITPM rate limit. Only `input_tokens` and `cache_creation_input_tokens` count. This allows heavily cached workloads to process far more total tokens per minute.

Q29 BATCH PROCESSING

What is the cost savings for batch processing compared to standard API requests?

- A 25%
- B 50%**
- C 75%
- D No savings, same price

Correct: B. Batch processing provides 50% cost savings compared to standard synchronous API requests, at the trade-off of higher latency (up to 24 hours).

Q30 BATCH PROCESSING

What is the `processing_status` value that indicates a batch has finished?

- A `"completed"`
- B `"ended"`**
- C `"finished"`
- D `"done"`

Correct: B. The batch `processing_status` transitions through: `in_progress` → `ended` . Other possible statuses are `canceling` , `canceled` , and `expired` .

Q31 BATCH PROCESSING

How should you correlate batch results with the original requests?

- A By the order results appear in the response
- B By the `custom_id` field on each result**
- C By the `message.id` of each response
- D By sequential numbering starting from 0

Correct: B. Results arrive in any order (not the order submitted). Each result carries the `custom_id` you set in the request. Always key by `custom_id` , never by position.

Q32 BATCH PROCESSING

What is the maximum number of requests allowed in a single batch?

- A 1,000
- B 10,000
- C 100,000**
- D 1,000,000

Correct: C. A single batch can contain up to 100,000 requests. Batches have a 24-hour processing window before results expire.

Q33 BATCH PROCESSING

Which of the following is NOT a valid batch result type?

- A succeeded
- B errored
- C timed_out**
- D expired

Correct: C. The valid batch result types are `succeeded` , `errored` , `canceled` , and `expired` . There is no `timed_out` type — requests that exceed the 24-hour window are marked as `expired` .

Q34 THINKING

What thinking configuration should you use for Claude Opus 5?

- A `{"type": "enabled", "budget_tokens": 8000}`
- B `{"type": "adaptive"}`**
- C `{"type": "enabled"}`
- D `{"type": "standard"}`

Correct: B. On Opus 4.6+ and all current models (except Haiku 4.5), use `{"type": "adaptive"}` . `budget_tokens` is removed on Opus 4.7+ and returns a 400 error. The legacy `{"type": "enabled", "budget_tokens": N}` is only for Haiku 4.5.

Q35 THINKING

Where is the `effort` parameter placed in the API request?

- A Top-level, alongside `model` and `max_tokens`
- B Inside the `thinking` object
- C Inside `output_config` : `output_config: {"effort": "high"}`**
- D As a query parameter on the URL

Correct: C. The effort parameter is placed inside `output_config` , not top-level and not inside `thinking` . Valid levels: `low` , `medium` , `high` (default for most models), `xhigh` , `max` .

Q36 THINKING

What must you preserve when passing thinking blocks back in a multi-turn conversation?

- A Only the thinking text
- B Only the thinking block type
- C The entire thinking block unchanged, including the `signature` field**
- D Nothing — thinking blocks should be removed before the next turn

Correct: C. Thinking blocks must be passed back unchanged, including their `signature` field. Never modify the thinking text or signature. Thinking blocks from a different model are silently dropped.

Q37 THINKING

How are thinking tokens billed?

- A As input tokens at the input rate
- B As output tokens at the output rate, counting toward `max_tokens`**
- C They are free and not billed
- D At a special reduced "thinking" rate

Correct: B. Thinking tokens are classified as output tokens and billed at the output rate. They also count toward the `max_tokens` limit. With `display: "omitted"`, thinking still occurs and is billed, but the text field is empty.

Q38 THINKING

What happens if you send `thinking: {"type": "disabled"}` to Claude Fable 5.1?

- A Thinking is disabled and the model generates text directly
- B The API returns a 400 error**
- C Thinking is disabled but with reduced output quality
- D The parameter is silently ignored

Correct: B. On Fable 5 / Fable 5.1, thinking is always on. Explicit `{"type": "disabled"}` returns a 400 error. To reduce thinking depth, use lower `effort` levels instead of disabling it.

Q39 RATE LIMITS

What are the three types of rate limits enforced by the Claude API?

- A Requests per second, tokens per second, connections per second
- B Requests per minute (RPM), input tokens per minute (ITPM), output tokens per minute (OTPM)**
- C Requests per hour, tokens per hour, cost per hour
- D Concurrent requests, total tokens, daily budget

Correct: B. The three rate limit types are RPM (Requests Per Minute), ITPM (Input Tokens Per Minute), and OTPM (Output Tokens Per Minute).

Q40 RATE LIMITS

Which HTTP status code indicates you have hit a rate limit?

- A 400
- B 403
- C 429**
- D 503

Correct: C. HTTP 429 is the rate limit error code. The response includes a `retry-after` header indicating when to retry. The SDKs automatically retry 429s with exponential backoff.

Q41 RATE LIMITS

How does the `enforced_spend_limit_reached` error differ from a rate limit error?

- A It uses HTTP 429 but with a different error code
- B It is a 400-level error indicating the monthly spending cap has been hit (not retryable)**
- C It is identical to a rate limit error
- D It uses HTTP 503 and requires waiting 24 hours

Correct: B. Spend limit errors are 400-level errors, not 429s. They indicate you have hit your monthly spending cap. The solution is to increase the spend limit in the Console, not to retry the request.

Q42 RATE LIMITS

Which response header tells you how many requests remain in your current rate limit window?

- A `x-ratelimit-remaining`
- B `anthropic-ratelimit-requests-remaining`**
- C `rate-limit-remaining`
- D `x-api-remaining-requests`

Correct: B. The Anthropic API uses `anthropic-ratelimit-*` prefixed headers: `-requests-limit` , `-requests-remaining` , `-requests-reset` , and similar for input-tokens and output-tokens.

Q43 RATE LIMITS

What does HTTP status code 529 indicate?

- A Authentication failure
- B Invalid request parameters
- C Resource not found
- D API is temporarily overloaded (retryable)**

Correct: D. HTTP 529 is a non-standard code used by the Claude API for `overloaded_error` . The API is temporarily at capacity. Retry with exponential backoff.

Q44 RATE LIMITS

Why is it a best practice to catch specific exception types in a chain rather than a single broad `APIStatusError` ?

- A Broad exceptions are not supported by the SDK
- B A single broad catch loses the distinction between retryable (429, 5xx) and non-retryable (400, 404) failures**
- C Broad exceptions cause performance issues
- D The SDK does not support catching `APIStatusError`

Correct: B. Catching `NotFoundError` → `RateLimitError` → `APIStatusError` → `APIConnectionError` lets you handle each failure mode appropriately (retry vs. fix vs. escalate).

Q45 TESTING & PRODUCTION

When using an LLM as an evaluator (LLM judge), which model should you use for evaluation?

- A** The same model being evaluated
- B** A different model than the one being evaluated
- C** Always use the cheapest model available
- D** Always use the most expensive model available

Correct: B. Using a different model for evaluation prevents self-bias, where the same model rates its own outputs favorably. For example, evaluate Opus outputs with Sonnet, or vice versa.

Q46 TESTING & PRODUCTION

Where should API keys be stored in a production application?

- A** Hardcoded in the application source code
- B** In a `.env` file committed to Git
- C** In environment variables or a secrets manager, never in source code or Git
- D** In the client-side JavaScript bundle

Correct: C. API keys should be stored in environment variables or a dedicated secrets manager. Never commit keys to Git, embed them in client-side code, or log them.

Q47 TESTING & PRODUCTION

What does Zero Data Retention (ZDR) mean for API requests?

- A** Responses are encrypted at rest
- B** The API does not log request metadata
- C** Input and output data is not stored or used for model training after the request completes
- D** API keys are rotated automatically

Correct: C. Zero Data Retention means that request inputs and outputs are not persisted by Anthropic after the request completes, and are not used for model training. This is important for handling sensitive data.

Q48 TESTING & PRODUCTION

What is the role of Anthropic when you use the Claude API directly?

- A** Data controller
- B** Data processor (your organization is the controller)
- C** Data owner
- D** Joint controller with your organization

Correct: B. When using the Claude API, Anthropic acts as the data processor. Your organization is the data controller, responsible for determining the purposes and means of processing.

Q49 TESTING & PRODUCTION

Which cost optimization strategy provides the largest single-lever reduction for most workloads?

- A** Switching to a cheaper model
- B** Prompt caching (cache reads at 0.1x)
- C** Reducing `max_tokens`
- D** Using batch processing

Correct: B. Prompt caching is typically the largest single cost reduction lever because cache reads cost only 0.1x the standard input price, and most applications send the same system prompt and tool definitions on every request. It also does not reduce quality, unlike model downgrading or effort reduction.

Q50 TESTING & PRODUCTION

What is the purpose of workspaces in the Anthropic Console?

- A** They provide different model versions for A/B testing
- B** They are used to organize conversation history
- C** They provide team isolation with independent API keys, rate limits, and usage tracking
- D** They enable faster model inference for premium tiers

Correct: C. Workspaces provide team isolation with independent API keys, per-workspace rate limits, separate usage tracking and billing. They are useful for separating production, staging, and development environments.

Anthropic Claude Developer Certification Study Guide

Compiled from official documentation at platform.claude.com/docs

This guide is for educational purposes. Always refer to the latest official documentation for the most current information.

Last updated: September 2026